

实验一函数说明

(一) CreateProcess 函数

CreateProcess 用来创建一个新的进程和它的主线程，这个新进程运行指定的可执行文件。

```
BOOL CreateProcess(LPCTSTR lpApplicationName,  
                  LPTSTR lpCommandLine,  
                  LPSECURITY_ATTRIBUTES lpProcessAttributes,  
                  LPSECURITY_ATTRIBUTES lpThreadAttributes,  
                  BOOL bInheritHandles,  
                  DWORD dwCreationFlags,  
                  LPVOID lpEnvironment,  
                  LPCTSTR lpCurrentDirectory,  
                  LPSTARTUPINFO lpStartupInfo,  
                  LPPROCESS_INFORMATION lpProcessInformation);
```

参数

- **lpApplicationName**: 指定新进程要使用的可执行文件的名称
- **lpCommandLine**: 指定要传给新进程的命令行字符串。
- **lpProcessAttributes**: 进程内核对象安全属性，若为 NULL，将指定默认的安全描述符；
- **lpThreadAttributes**: 主线程内核对象安全属性，若为 NULL，将指定默认的安全描述符；
- **bInheritHandles**: 指定子进程是否能继承父进程中可继承的内核对象，若为 TRUE 则可继承，若为 FALSE 则不可继承
- **dwCreationFlags**: 标识了影响新进程创建方式的标志（flag）。
 - CREATE_NEW_CONSOLE: 为新进程创建一个新的控制台窗口。
- **lpEnvironment**: 指向一块内存，其中包含新进程要使用的环境字符串
- **lpCurrentDirectory**: 子进程的当前驱动器和目录
- **lpStartupInfo**: 指向一个 STARTUPINFO 结构，通常情况下是使用默认即可（将所有成员初始化为 0，并将 cb 成员设为此结构的大小）

- **lpProcessInformation**: [out], 指向一个 PROCESS_INFORMATION 结构, 函数在返回之前, 会初始化这个结构体。

返回值

- CreateProcess 在进程完全初始化好之前就返回 TRUE。
- 若失败则返回 FALSE

(二) lpStartupInfo

```
typedef struct _STARTUPINFO
{
    DWORD cb;
    LPTSTR lpReserved;
    LPTSTR lpDesktop;
    LPTSTR lpTitle;
    DWORD dwX;
    DWORD dwY;
    DWORD dwXSize;
    DWORD dwYSize;
    DWORD dwXCountChars;
    DWORD dwYCountChars;
    DWORD dwFillAttribute;
    DWORD dwFlags;
    WORD wShowWindow;
    WORD cbReserved2;
    LPBYTE lpReserved2;
    HANDLE hStdInput;
    HANDLE hStdOutput;
    HANDLE hStdError;
} STARTUPINFO, *LPSTARTUPINFO;
```

指向一个用于决定新进程的主窗体如何显示的 STARTUPINFO 结构体。

大多数应用程序都希望生成的应用程序只是使用默认值，最起码要全部初始化为 0，再把 cb 成员设为此结构体的大小，如果没有清 0，则新进程可能创建失败

(三) PROCESS_INFORMATION 结构

```
typedef struct _PROCESS_INFORMATION{  
    HANDLE hProcess;  
    HANDLE hThread;  
    DWORD dwProcessId;  
    DWORD dwThreadId;  
}PROCESS_INFORMATION;
```

其中成员含义如下。

- ① hProcess：返回新进程的句柄。
- ② hThread：返回[主线程](#)的句柄。
- ③ dwProcessId：返回一个全局进程[标识符](#)。该标识符用于标识一个进程。从进程被创建到终止，该值始终有效。
- ④ dwThreadId：返回一个全局[线程](#)标识符。该标识符用于标识一个线程。从线程被创建到终止，该值始终有效。

(四) HANDLE ([句柄](#))

HANDLE ([句柄](#)) 是 Windows 操作系统中的一个概念。在 Windows 程序中，有各种各样的资源（窗口、图标、光标等），系统在创建这些资源时会为它们分配内存，并返回标示这些资源的标示号，即句柄。[句柄指的是一个核心对象在某一个进程中的唯一索引](#)，而不是[指针](#)。由于[地址空间](#)的限制，句柄所标识的内容对进程是不可见的，只能由操作系统通过进程句柄列表来进行维护。

句柄列表：每个进程都要创建一个句柄列表，这些句柄指向各种系统资源，比如[信号量](#)，[线程](#)，和文件等，进程中的所有线程都可以访问这些资源。

在 windows 中，句柄是和对象一一对应的 32 位无符号整数值。对象可以映射到唯一的句柄，句柄也可以映射到唯一的对象。

(五) OpenProcess

用来打开一个已存在的进程对象，并返回进程的句柄。

1.函数原型

```
HANDLE OpenProcess(  
    DWORD dwDesiredAccess, //渴望得到的访问权限（标志）  
    BOOL bInheritHandle, // 是否继承句柄  
    DWORD dwProcessId // 进程标示符  
);
```

2.返回值:

如成功，返回值为指定进程的句柄。

如失败，返回值为空，可调用 GetLastError 获得错误代码。

(六) TerminateProcess 函数

这个函数可以用来终止或者说杀死一个进程，它不会留给进程及其所有线程清理的时间，系统会马上终止(杀死)这个进程的所有线程，致使进程终止。在使用此函数前我们必须调用 [OpenProcess 函数](#)来获得我们要终止(杀死)进程的句柄，并且要获得进程的 PROCESS_TERMINATE 权限。

函数原型:

```
BOOL TerminateProcess(HANDLE hProcess,UINT uExitCode)
```

参数说明:

1、hProcess: 要终止(杀死)进程的句柄，需要有 PROCESS_TERMINATE 权限。

2、uExitCode: 设置进程的退出值。可通过 GetExitCodeProcess 函数得到一个进程的退出值。

返回值:

如果失败将返回 FALSE(0)，而成功将返回一个非零值。

注: 不要用 if(ret==TRUE)去检测函数是否调用成功，因为函数调用成功会返回一个非零值，但不一定是 TRUE(1)